Les embeddings, ou comment synthétiser l'information textuelle

Lino Galiana
Version du 2 novembre 2026

Pour pouvoir utiliser des données textuelles dans des algorithmes de *machine learning*, il faut les vectoriser, c'est à dire transformer le texte en données numériques. Dans ce TP, nous allons comparer différentes méthodes de vectorisation, à travers une tâche de prédiction : *peut-on prédire un auteur littéraire à partir d'extraits de ses textes ?* Parmi ces méthodes, on va notamment explorer le modèle `Word2Vec`, qui permet d'exploiter les structures latentes d'un texte en construisant des *word embeddings* (plongements de mots).

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **1**
1.1 Données utilisées 2
1.2 Packages à installer 2
2 Préparation des données **3**
2.1 Preprocessing 3
2.2 Encodage de la variable à prédire 5
2.3 Construction des bases d'entraînement et de test 5
3 Vectorisation par l'approche *bag of words* **5**
3.1 Trouver le texte le plus similaire 22
3.2 Trouver l'auteur le plus proche: une introduction au classifieur naif Bayes 25
4 Le modèle Word2Vec, une représentation plus synthétique **38**
4.1 Vers une représentation plus synthétique du langage 38
4.2 Des relations sémantique entre les termes 39
4.3 Comment ces modèles sont-ils entraînés ? 41
4.4 Modèles liés 42
5 Les *transformers*: une représentation plus riche du langage **43**
Informations additionnelles 45
Bibliographie **47**

> ⚠️ Warning
>
> Ce chapitre va évoluer prochainement.

1 Introduction

Cette page approfondit certains aspects présentés dans la partie introductive. Nous allons avancer dans notre compréhension des enjeux du NLP grâce à la modélisation du langage.

Nous partirons de la conclusion, évoquée à la fin du précédent chapitre, que les approches fréquentistes présentent plusieurs inconvénients : représentation du langage sur des régularités statistiques indépendantes des proximités entre des mots ou phrases, difficulté à prendre en compte le contexte.

L'objectif de ce chapitre est d'évoquer le premier point. Il s'agira d'une introduction au sujet des *embeddings*, ces représentations du langage qui sont au fondement des modèles de langage actuels utilisés par des outils entrés dans notre quotidien (`DeepL`, `ChatGPT`...).

1.1 Données utilisées

Nous allons continuer notre exploration de la littérature avec, à nouveau, les trois auteurs anglophones :

- Edgar Allan Poe, (EAP) ;
- HP Lovecraft (HPL) ;
- Mary Wollstonecraft Shelley (MWS).

Les données sont disponibles sur un CSV mis à disposition sur `Github`. L'URL pour les récupérer directement est https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv.

Pour rentrer dans le sujet des *embeddings*, nous allons faire de la modélisation du langage en essayant de prédire les auteurs ayant écrit tel ou tel texte. On parle de modèle de langage pour désigner la représentation d'un texte ou d'une langue sous la forme d'une distribution de probabilités de termes (généralement les mots).

> **i** Sources d'inspiration
>
> Ce chapitre s'inspire de plusieurs ressources disponibles en ligne:
>
> - Un premier *notebook* sur `Kaggle` et un deuxième ;
> - Un dépôt `Github` ;

1.2 Packages à installer

Comme dans la partie précédente, il faut télécharger des librairies spécialiséees pour le NLP, ainsi que certaines de leurs dépendances. Ce TD utilisera plusieurs librairies dont certaines dépendent de `PyTorch` qui est une librairie volumineuse.

> **!** PyTorch sur le SSPCloud
>
> **La prochaine remarque ne concerne que les utilisateurs.trices du `SSPCloud`.**
>
> Les services `Python` standards sur le `SSPCloud` (les services `vscode-python` et `jupyter-python`) ne proposent pas `PyTorch` préinstallé. Cette librairie est en effet assez volumineuse (de l'ordre de 600Mo) et nécessite un certain nombre de configurations *ad hoc* pour fonctionner de manière transparente quelle que soit la configuration logicielle derrière. Pour des raisons de frugalité écologique, cet environnement *boosté* n'est pas proposé par défaut. Néanmoins, si besoin, un tel environnement où `Pytorch` est pré à l'emploi est disponible.
>
> Pour cela, il suffit de démarrer un service `vscode-pytorch` ou `jupyter-pytorch`. Si vous avez utilisé l'un des boutons disponibles ci-dessus, c'est ce service standardisé qui a automatiquement été mis à disposition pour vous.

```
!pip install numpy pandas spacy transformers scikit-learn langchain_community
```

Ensuite, comme nous allons utiliser la librairie `SpaCy` avec un corpus de textes en Anglais, il convient de télécharger le modèle NLP pour l'Anglais. Pour cela, on peut se référer à la documentation de `SpaCy`, extrêmement bien faite.

```
!python -m spacy download en_core_web_sm
```

2 Préparation des données

Nous allons à nouveau utiliser le jeu de données `spooky` :

```
import pandas as pd

data_url  =  'https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.
csv'
spooky_df = pd.read_csv(data_url)
```

Le jeu de données met ainsi en regard un auteur avec une phrase qu'il a écrite :

```
spooky_df.head()
```

	id	text	author
0	id26305	This process, however, afforded me no means of...	EAP
1	id17569	It never once occurred to me that the fumbling...	HPL
2	id11008	In his left hand was a gold snuff box, from wh...	EAP
3	id27763	How lovely is spring As we looked from Windsor...	MWS
4	id12958	Finding nothing else, not even gold, the Super...	HPL

2.1 Preprocessing

Comme nous l'avons évoqué dans le chapitre précédent, la première étape de tout travail sur données textuelles est souvent celle du *preprocessing*, qui inclut notamment les étapes de tokenization et de nettoyage du texte.

On se contentera ici d'un *preprocessing* minimaliste : suppression de la ponctuation et des *stop words* (pour la visualisation et les méthodes de vectorisation basées sur des comptages).

Pour initialiser le processus de nettoyage, on va utiliser le corpus `en_core_web_sm` de `Spacy`

```
import spacy
nlp = spacy.load('en_core_web_sm')
```

On va utiliser un `pipe` `spacy` qui permet d'automatiser, et de paralléliser, un certain nombre d'opérations. Les *pipes* sont l'équivalent, en NLP, de nos *pipelines* `scikit` ou des *pipes* `pandas`. Il s'agit donc d'un outil très approprié pour industrialiser un certain nombre d'opérations de *preprocessing* :

```
from typing import List
import spacy

def clean_docs(
    texts: List[str],
```

```
    remove_stopwords: bool = False,
    n_process: int = 4,
    remove_punctuation: bool = True
) -> List[str]:
    """
    Cleans a list of text documents by tokenizing, optionally removing stopwords, and optionally
removing punctuation.

    Parameters:
        texts (List[str]): List of text documents to clean.
        remove_stopwords (bool): Whether to remove stopwords. Default is False.
        n_process (int): Number of processes to use for processing. Default is 4.
        remove_punctuation (bool): Whether to remove punctuation. Default is True.

    Returns:
        List[str]: List of cleaned text documents.
    """
    # Load spacy's nlp model
    docs = nlp.pipe(
        texts,
        n_process=n_process,
        disable=['parser', 'ner', 'lemmatizer', 'textcat']
    )

    # Pre-load stopwords for faster checking
    stopwords = set(nlp.Defaults.stop_words)

    # Process documents
    docs_cleaned = (
        ' '.join(
            tok.text.lower().strip()
            for tok in doc
            if (not remove_punctuation or not tok.is_punct) and
               (not remove_stopwords or tok.text.lower() not in stopwords)
        )
        for doc in docs
    )

    return list(docs_cleaned)
```

On applique la fonction `clean_docs` à notre colonne `pandas`. Les `pandas.Series` étant itérables, elles se comportent comme des listes et fonctionnent ainsi très bien avec notre `pipe spacy`.

```
spooky_df['text_clean'] = clean_docs(spooky_df['text'])
```

```
spooky_df.head()
```

	id	text	author	text_clean
0	id26305	This process, however, afforded me no means of…	EAP	this process however afforded me no means of a…
1	id17569	It never once occurred to me that the fumbling…	HPL	it never once occurred to me that the fumbling…
2	id11008	In his left hand was a gold snuff box, from wh…	EAP	in his left hand was a gold snuff box from whi…

	id	text	author	text_clean
3	id27763	How lovely is spring As we looked from Windsor…	MWS	how lovely is spring as we looked from windsor…
4	id12958	Finding nothing else, not even gold, the Super…	HPL	finding nothing else not even gold the superin…

2.2 Encodage de la variable à prédire

On réalise un simple encodage de la variable à prédire : il y a trois catégories (auteurs), représentées par des entiers 0, 1 et 2. Pour cela, on utilise le `LabelEncoder` de `Scikit` déjà présenté dans la partie modélisation. On va utiliser la méthode `fit_transform` qui permet, en un tour de main, d'appliquer à la fois l'entraînement (`fit`), à savoir la création d'une correspondance entre valeurs numériques et *labels*, et l'appliquer (`transform`) à la même colonne.

On peut vérifier les classes de notre `LabelEncoder` :

```
array(['EAP', 'HPL', 'MWS'], dtype=object)
```

2.3 Construction des bases d'entraînement et de test

On met de côté un échantillon de test (20 %) avant toute analyse (même descriptive). Cela permettra d'évaluer nos différents modèles toute à la fin de manière très rigoureuse, puisque ces données n'auront jamais utilisées pendant l'entraînement.

Notre échantillon initial n'est pas équilibré (*balanced*) : on retrouve plus d'oeuvres de certains auteurs que d'autres. Afin d'obtenir un modèle qui soit évalué au mieux, nous allons donc stratifier notre échantillon de manière à obtenir une répartition similaire d'auteurs dans nos ensembles d'entraînement et de test.

```
from sklearn.model_selection import train_test_split

y = spooky_df["author"]
X = spooky_df['text_clean']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Aperçu du premier élément de `X_train` :

```
'this process however afforded me no means of ascertaining the dimensions of my dungeon as i
might make its circuit and return to the point whence i set out without being aware of the fact
so perfectly uniform seemed the wall'
```

3 Vectorisation par l'approche *bag of words*

La représentation de nos textes sous forme de sac de mot permet de vectoriser notre corpus et ainsi d'avoir une représentation numérique de chaque texte. On peut à partir de là effectuer plusieurs types de tâches de modélisation.

Définissons notre représentation vectorielle par TF-IDF grâce à `Scikit`:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline

pipeline_tfidf = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=10000)),
])
pipeline_tfidf
```

	steps steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `transform`. See :ref:`Combining Estimators <combining_estimators>` for more details.	[('tfidf', ...)]
	transform_input transform_input: list of str, default=None The names of the :term:`metadata` parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to ``fit`` (other than ``X``) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:`metadata routing <metadata_routing>`. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using ``sklearn.set_config(enable_metadata_routing=True)``. .. versionadded:: 1.6	None
	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last	None

	step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute ``named_steps`` or ``steps`` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:`sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py` for an example on how to enable caching.	
	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False
	max_features max_features: int, default=None **If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus.** **Otherwise, all features are used.** **This parameter is ignored if vocabulary is not None.**	**10000**
	input input: {'filename', 'file', 'content'}, default='content' - If `'filename'`, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If `'file'`, the sequence items must have a 'read' method (file-like	'content'

	object) that is called to fetch the bytes in memory. - If `'content'`, the input is expected to be a sequence of items that can be of type string or byte.	
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	'strict'
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing	None

	(string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	
	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``'filename'`` or ``'file'``, the data is first read from the file and then passed to the given callable analyzer.	'word'
	stop_words stop_words: {'english'}, list, default=None If a string, it is passed to _check_stop_list and the appropriate stop list is returned. 'english' is currently the only supported string value. There are several known issues with 'english' and you should consider an alternative (see :ref:`stop_words`). If a list, that list is assumed to	None

	contain stop words, all of which will be removed from the resulting tokens. Only applies if ``analyzer == 'word'``. If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	
	token_pattern token_pattern: str, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different n-grams to be extracted. All values of n such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means only bigrams.	(1, ...)

	Only applies if ``analyzer`` is not callable.	
	max_df max_df: float or int, default=1.0 When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1.0
	min_df min_df: float or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents.	None
	binary binary: bool, default=False If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set `binary` to True,	False

	`use_idf` to False and `norm` to None to get 0/1 outputs).	
	dtype dtype: dtype, default=float64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.float64'>
	norm norm: {'l1', 'l2'} or None, default='l2' Each output row will have unit norm, either: - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied. - 'l1': Sum of absolute values of vector elements is 1. See :func:`~sklearn.preprocessing.normalize`. - None: No normalization.	'l2'
	use_idf use_idf: bool, default=True Enable inverse-document-frequency reweighting. If False, idf(t) = 1.	True
	smooth_idf smooth_idf: bool, default=True Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.	True
	sublinear_tf sublinear_tf: bool, default=False Apply sublinear tf scaling, i.e. replace tf with 1 + log(tf).	False

Entraînons d'ores et déjà notre modèle à vectoriser le texte à partir de la méthode TF-IDF. Pour le moment il n'est pas encore question de faire de l'évaluation, faisons donc un entraînement sur l'ensemble de notre base et pas seulement sur `X_train`.

```
pipeline_tfidf.fit(spooky_df['text_clean'])
```

	steps steps: list of tuples **List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `transform`. See :ref:`Combining Estimators <combining_estimators>` for more details.**	**[('tfidf', ...)]**
	transform_input transform_input: list of str, default=None The names of the :term:`metadata` parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to ``fit`` (other than ``X``) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:`metadata routing <metadata_routing>`. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using ``sklearn.set_config(enable_metadata_routing=True)``. .. versionadded:: 1.6	None
	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the	None

	transformer instance given to the pipeline cannot be inspected directly. Use the attribute ``named_steps`` or ``steps`` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:`sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py` for an example on how to enable caching.	
	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False
	max_features max_features: int, default=None **If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used.** **This parameter is ignored if vocabulary is not None.**	10000
	input input: {'filename', 'file', 'content'}, default='content' - If ``'filename'``, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If ``'file'``, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory. - If ``'content'``, the input is expected to be a sequence of items that can be of type string or byte.	'content'

	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	'strict'
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	None

	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``'filename'`` or ``'file'``, the data is first read from the file and then passed to the given callable analyzer.	'word'
	stop_words stop_words: {'english'}, list, default=None If a string, it is passed to _check_stop_list and the appropriate stop list is returned. 'english' is currently the only supported string value. There are several known issues with 'english' and you should consider an alternative (see :ref:`stop_words`). If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if ``analyzer == 'word'``.	None

	If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	
	token_pattern token_pattern: str, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different n-grams to be extracted. All values of n such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means only bigrams. Only applies if ``analyzer`` is not callable.	(1, ...)
	max_df max_df: float or int, default=1.0 When building the vocabulary ignore terms that have a docu-	1.0

	ment frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	
	min_df min_df: float or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents.	None
	binary binary: bool, default=False If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set `binary` to True, `use_idf` to False and `norm` to None to get 0/1 outputs).	False
	dtype dtype: dtype, default=float64	<class 'numpy.float64'>

	Type of the matrix returned by fit_transform() or transform().	
	norm norm: {'l1', 'l2'} or None, default='l2' Each output row will have unit norm, either: - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied. - 'l1': Sum of absolute values of vector elements is 1. See :func:`~sklearn.preprocessing.normalize`. - None: No normalization.	'l2'
	use_idf use_idf: bool, default=True Enable inverse-document-frequency reweighting. If False, idf(t) = 1.	True
	smooth_idf smooth_idf: bool, default=True Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.	True
	sublinear_tf sublinear_tf: bool, default=False Apply sublinear tf scaling, i.e. replace tf with 1 + log(tf).	False

Name	Type	Value
fixed_vocabulary_ fixed_vocabulary_: bool True if a fixed vocabulary of term to indices mapping is provided by the user.	bool	False
idf_ idf_: array of shape (n_features,) The inverse document frequency (IDF) vector; only defined if ``use_idf`` is True.	ndarray[float64](10000,)	[8.4 ,7.48,9.5 ,...,8.8 ,9.09,9.27]

vocabulary_ vocabulary_: dict A mapping of terms to feature indices.	dict	{'abandon': np.int64(0), 'ab…ed': np.int64(1), 'ab…ng': np.int64(2), 'ab…nt': np.int64(3), …}

abandon
abandoned
abandoning
abandonment
abaout
abbey
abdication
abdul
abernethy
abhor
abhorred
abhorrence
abhorrent
abilities
ability
abject
able
abnormal
abnormality
abnormally
abode
abodes
about
above
abroad
abrupt
abruptly
absence
absent
absolute
absolutely
absorbed
absorbing
abstract
abstraction
absurd
absurdity
absurdly
abundance

abundant
abundantly
abysmal
abyss
abysses
academy
accelerated
accent
accents
accept
accepted
access
accident
accidental
accidentally
accidents
accommodation
accompanied
accompany
accompanying
accomplish
accomplished
accomplishing
accomplishment
accomplishments
accord
accordance
accorded
according
accordingly
accost
accosted
account
accounted
accounting
accounts
accumulated
accuracy
accurate
accurately
accursed
accuse
accused

accustomed
acetylene
achieve
achieved
achievement
achievements
aching
acknowledge
acknowledged
acquaintance
acquaintances
acquainted
acquire
acquired
acquirement
acquiring
acre
across

3.1 Trouver le texte le plus similaire

En premier lieu, on peut chercher le texte le plus proche, au sens de TF-IDF, d'une phrase donnée. Prenons cet exemple:

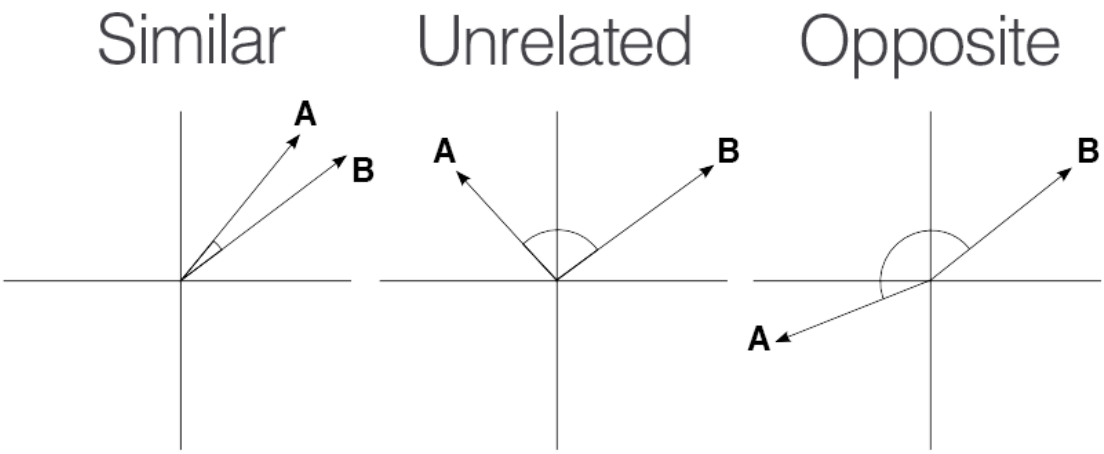
```
text = "He was afraid by Frankenstein monster"
```

Comment retrouver le texte le plus proche de celui-ci ? Il faudrait transformer notre texte dans cette même représentation vectorielle et rapprocher ensuite celui-ci des autres textes représentés sous cette même forme.

Cela revient à effectuer une tâche de recherche d'information, cas d'usage classique du NLP, mis en oeuvre par exemple par les moteurs de recherche. Le terme Frankenstein étant assez discrminant, nous devrions, grâce à TF-IDF, retrouver des similarités entre notre texte et d'autres textes écrits par Mary Shelley.

Une métrique régulièrement utilisée pour comparer des vecteurs est la similarité cosinus. Il s'agit d'ailleurs d'une mesure centrale dans l'approche moderne du NLP. Celle-ci a plus de sens avec des vecteurs dense, que nous verrons prochainement, qu'avec des vecteurs comportant beaucoup de 0 comme le sont les vecteurs *sparse* des mesures *bag-of-words*. Néanmoins c'est déjà un exercice intéressant pour comprendre la similarité entre deux vecteurs.

Si chaque dimension d'un vecteur correspond à une direction, l'idée derrière la similarité cosinus est de mesurer l'angle entre deux vecteurs. L'angle sera réduit si les vecteurs sont proches.



3.1.a Avec `Scikit-Learn`

Exercice 1: recherche de similarité avec TF-IDF

1. Utiliser la méthode `transform` pour vectoriser tout notre corpus d'entraînement.
2. En supposant que votre jeu d'entraînement vectorisé s'appelle `X_train_tfidf`, vous pouvez le transformer en *DataFrame* avec la commande suivante:

```
X_train_tfidf = pd.DataFrame(
    X_train_tfidf.todense(), columns=pipeline_tfidf.get_feature_names_out()
)
```

3. Utiliser la méthode `cosine_similarity` de `Scikit` pour calculer la similarité cosinus entre notre texte vectorisé et l'ensemble du corpus d'entraînement grâce au code suivant:

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

cosine_similarities = cosine_similarity(
    X_train_tfidf,
    pipeline_tfidf.transform([text])
).flatten()

top_4_indices = np.argsort(cosine_similarities)[-4:][::-1]  # Tri décroissant
top_4_similarities = cosine_similarities[top_4_indices]
```

4. Retrouver les documents concernés. Êtes-vous satisfait du résultat ? Comprenez-vous ce qu'il s'est passé ?

A l'issue de l'exercice, les 4 textes les plus similaires sont:

	text	author	score
8181	Listen to me, Frankenstein.	MWS	0.402964
8606	He was gazing at me gaspingly and fascinatedly...	HPL	0.330177
14550	The light is dimmer and the gods are afraid. . .	HPL	0.314670
11366	I screamed aloud that I was not afraid; that I...	HPL	0.311235

3.1.b Avec `Langchain`

Cette approche de calcul de similarité textuelle est assez laborieuse avec `Scikit`. Avec le développement continu d'applications `Python` utilisant des modèles de langage, un écosystème très complet s'est développé pour pouvoir faire ces tâches en quelques lignes de code avec `Python`.

Parmi les outils les plus précieux, nous trouvons `Langchain`, un écosystème `Python` haut-niveau permettant de construire des chaînes de production utilisant des données textuelles.

Nous allons ici procéder en 2 étapes:

- Créer un *retriever*, c'est-à-dire vectoriser avec TF-IDF notre corpus (les textes de nos trois auteurs) et les stocker sous forme de base de données vectorielle.
- Vectoriser à la volée notre texte de recherche (l'objet `text` créé précédemment) et rechercher sa contrepartie la plus proche dans la base de données vectorielle.

La vectorisation de notre corpus se fait très simplement grâce à `Langchain` puisque le `TfidfVectorizer` de `Scikit` est encapsulé dans un module *ad hoc* de `Langchain`

```
from langchain_community.retrievers import TFIDFRetriever
from langchain_community.document_loaders import DataFrameLoader

loader = DataFrameLoader(spooky_df, page_content_column="text_clean")

retriever = TFIDFRetriever.from_documents(
    loader.load()
)
```

```
/tmp/ipykernel_338/3052793950.py:1: DeprecationWarning: `langchain-community` is being sunset
and is no longer actively maintained. See https://github.com/langchain-ai/langchain-community/
issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.retrievers import TFIDFRetriever
```

Cet objet `retriever` est un point d'entrée sur notre corpus. `Langchain` présente l'intérêt de fournir plusieurs points d'entrées standardisés, forts utiles dans les projets de NLP puisqu'il suffit de changer les vectoriseurs en entrée sans avoir à changer leur usage en fin de chaîne.

La méthode `invoke` permet de rechercher les vecteurs les plus similaires à notre texte de recherche:

```
retriever.invoke(text)
```

```
[Document(metadata={'id': 'id12587', 'text': 'Listen to me, Frankenstein.', 'author': 'MWS',
'author_encoded': 2}, page_content='listen to me frankenstein'),
 Document(metadata={'id': 'id09284', 'text': 'I screamed aloud that I was not afraid; that I never
could be afraid; and others screamed with me for solace.', 'author': 'HPL', 'author_encoded':
1}, page_content='i screamed aloud that i was not afraid that i never could be afraid and others
screamed with me for solace'),
 Document(metadata={'id': 'id09797', 'text': 'It seemed to be a sort of monster, or symbol
representing a monster, of a form which only a diseased fancy could conceive.', 'author': 'HPL',
'author_encoded': 1}, page_content='it seemed to be a sort of monster or symbol representing a
monster of a form which only a diseased fancy could conceive'),
 Document(metadata={'id': 'id10816', 'text': 'And, as I have implied, it was not of the dead man
himself that I became afraid.', 'author': 'HPL', 'author_encoded': 1}, page_content='and as i
have implied it was not of the dead man himself that i became afraid')]
```

La sortie est un objet `Langchain`, ce qui n'est pas pratique pour nous dans notre situation. On se ramène à un *DataFrame*:

```
documents = []
for best_echoes in retriever.invoke(text):
    documents += [{**best_echoes.metadata, **{"text_clean": best_echoes.page_content}}]

documents = pd.DataFrame(documents)
```

On peut ajouter à ce *DataFrame* la colonne de score:

On retrouve bien les mêmes documents:

	id	text	author	author_encoded	text_clean	score
0	id12587	Listen to me, Frankenstein.	MWS	2	listen to me frankenstein	0.402964
1	id09284	I screamed aloud that I was not afraid; that I...	HPL	1	i screamed aloud that i was not afraid that i ...	0.311235
2	id09797	It seemed to be a sort of monster, or symbol r...	HPL	1	it seemed to be a sort of monster or symbol re...	0.295587
3	id10816	And, as I have implied, it was not of the dead...	HPL	1	and as i have implied it was not of the dead m...	0.261818

> **i** La métrique BM25
>
> BM25 est un modèle de récupération d'informations basé sur la pertinence probabiliste, au même titre que TF-IDF. BM25 est souvent utilisée dans les moteurs de recherche pour classer les documents par rapport à une requête.
>
> BM25 repose sur une combinaison de la fréquence des termes (TF), la fréquence inverse des documents (IDF), et une normalisation basée sur la longueur des documents. Autrement dit, il s'agit de tenir compte d'améliorer TF-IDF tout en normalisant les mesures en fonction de la taille des *strings* afin de ne pas surpondérer les grands documents.
>
> BM25 est donc particulièrement performant dans des environnements où les documents varient en longueur et en contenu. C'est pour cette raison que des moteurs de recherche comme `Elasticsearch` en ont fait une pierre angulaire du mécanisme de recherche.

Pourquoi ne sont-ils pas tous pertinents ? On peut anticiper plusieurs raisons à cela.

La première hypothèse vient du fait qu'on entraîne notre vectoriseur sur un corpus biaisé. Certes Frankestein est un terme rare mais il est beaucoup plus fréquent dans notre corpus que dans la langue anglaise. L'*inverse document frequency* est donc biaisée en défaveur de ce terme: son apparition devrait être un signe beaucoup plus fort que le texte recherché correspond à Mary Shelley. Si cela peut améliorer un peu la pertinence des résultats renvoyés, ce n'est néanmoins pas là que le bât blesse.

L'approche fréquentiste suppose que les termes sont aussi dissemblables les uns que les autres. Une phrase où apparaît le terme *« créature »* ne bénéficiera pas d'un score positif si on recherche *« monstre »*. De plus, là encore, nous avons pris notre corpus comme un sac où les mots sont indépendants: on n'a pas plus de chance de tirer *« Frankenstein »* après *« docteur »*. Ces limites vont nous amener vers le sujet des *embeddings*. Néanmoins, si l'approche fréquentiste est un peu *old school*, elle n'est néanmoins pas inutile et représente souvent une *« tough to beat baseline »*. Dans les domaines de l'extraction d'information avec des textes courts, où chaque terme est porteur d'un signal fort, cette approche est souvent judicieuse.

3.2 Trouver l'auteur le plus proche: une introduction au classifieur naif Bayes

Avant d'explorer les *embeddings*, nous pouvons essayer d'avoir un cas d'usage un petit peu différent dans notre cadre probabiliste. Supposons qu'on désire maintenant faire de la prédiction d'auteur. Si l'intuition précédente est vraie - certains mots sont plus probables dans les textes de certains auteurs - cela veut dire qu'on peut entraîner un algorithme de classification automatique à prédire un auteur à partir d'un texte.

La méthode la plus naturelle pour se lancer dans cette approche est d'utiliser le classifieur naif de Bayes. Ce dernier est parfaitement adapté à l'approche fréquentiste que nous avons adoptée jusqu'à présent puisqu'il exploite les probabilités d'occurrence de mots par auteur.

Le classifieur naif de Bayes consiste à appliquer une règle de décision, à savoir sélectionner la classe la plus probable sachant la structure observée du document, c'est-à-dire les mots apparaissant dans celui-ci.

Autrement dit, on sélectionne la classe $\hat{c}$ qui est la plus probable, sachant les termes dans le document d.

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(c|d) = \arg\max_{c \in \mathcal{C}} \frac{\mathbb{P}(d|c)\mathbb{P}(c)}{\mathbb{P}(d)} \tag{1}$$

Comme ceci est classique en estimation bayésienne, on peut se passer de certains termes constants, à savoir $\mathbb{P}(d)$. La définition de la classe estimée peut ainsi être reformulée de cette manière:

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(d|c)\mathbb{P}(c) \tag{2}$$

L'hypothèse du sac de mot intervient à ce niveau. Un document d est une collection de mots w_i dont l'ordre n'a pas d'intérêt. Autrement dit, on peut se contenter de faire un modèle sur les mots, sans faire intervenir des probabilités conditionnelles sur l'ordre d'occurrence. La seconde hypothèse forte est l'hypothèse naive à laquelle la méthode doit son nom: la probabilité de tirer un mot ne dépend que de la catégorie c d'appartenance du document. Autrement dit, on peut considérer qu'un document est une suite de tirage indépendants de mots dont la probabilité ne dépend que de l'auteur.

Comme cela est expliqué dans la boite dédiée, en faisant ces hypothèses, on peut réécrire ce classifieur sous la forme

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(c) \prod_{w \in \mathcal{W}} \mathbb{P}(w|c)$$

avec $\mathcal{W}$ l'ensemble des mots dans le corpus (notre vocabulaire).

Empiriquement, nous sommes dans une tâche d'apprentissage supervisé où le *label* est la classe du document et les *features* sont nos mots vectorisés. Empiriquement, les probabilités sont estimées à partir du dénombrement des mots dans le corpus et des types de documents dans le corpus.

Il est bien-sûr possible de calculer toutes ces grandeurs à la main mais `Scikit` permet d'implémenter un estimateur naif de Bayes après avoir vectorisé son corpus comme le montre le prochain exercice. Cela peut néanmoins poser un problème pratique: en principe, le corpus de test ne doit pas comporter de nouveaux mots car ces « nouvelles » dimensions n'étaient pas présentes dans le corpus d'entraînement. En pratique, la solution la plus simple est souvent celle choisie: ces mots sont ignorés.

Exercice 2: le classifieur naif de Bayes

1. En repartant de l'exemple précédent, définir un *pipeline* qui vectorise chaque document (utiliser `CountVectorizer` plutôt que `TFIDFVectorizer`) et effectue une prédiction grâce à un modèle naif de Bayes.
2. Entraîner ce modèle, faire une prédiction sur le jeu de test.
3. Evaluer la performance de votre modèle
4. Faire une prédiction sur la phrase que nous avons utilisée tout à l'heure dans la variable `text`. Obtenez-vous ce qui était attendu ?
5. Regarder les probabilités obtenues (méthode `predict_proba`).

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```
pipeline = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('classifier', MultinomialNB())
])

# Train the pipeline on the training data
pipeline.fit(X_train, y_train)
```

	steps steps: list of tuples List of (name of step, estimator) tuples that are to be chained in sequential order. To be compatible with the scikit-learn API, all steps must define `fit`. All non-last steps must also define `transform`. See :ref:`Combining Estimators <combining_estimators>` for more details.	[('vectorizer', ...), ('classifier', ...)]
	transform_input transform_input: list of str, default=None The names of the :term:`metadata` parameters that should be transformed by the pipeline before passing it to the step consuming it. This enables transforming some input arguments to ``fit`` (other than ``X``) to be transformed by the steps of the pipeline up to the step which requires them. Requirement is defined via :ref:`metadata routing <metadata_routing>`. For instance, this can be used to pass a validation set through the pipeline. You can only set this if metadata routing is enabled, which you can enable using ``sklearn.set_config(enable_metadata_routing=True)``. .. versionadded:: 1.6	None
	memory memory: str or object with the joblib.Memory interface, default=None Used to cache the fitted transformers of the pipeline. The last	None

	step will never be cached, even if it is a transformer. By default, no caching is performed. If a string is given, it is the path to the caching directory. Enabling caching triggers a clone of the transformers before fitting. Therefore, the transformer instance given to the pipeline cannot be inspected directly. Use the attribute ``named_steps`` or ``steps`` to inspect estimators within the pipeline. Caching the transformers is advantageous when fitting is time consuming. See :ref:`sphx_glr_auto_examples_neighbors_plot_caching_nearest_neighbors.py` for an example on how to enable caching.	
	verbose verbose: bool, default=False If True, the time elapsed while fitting each step will be printed as it is completed.	False

Name	Type	Value
classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[<U3](3,)	['EAP','HPL','MWS']

	input input: {'filename', 'file', 'content'}, default='content' **- If ``'filename'``, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze.** **- If ``'file'``, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory.** **- If ``'content'``, the input is**	**'content'**

	expected to be a sequence of items that can be of type string or byte.	
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	'strict'
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (strip_accents and lowercase) stage while preserving the tokenizing and n-grams generation steps.	None

	Only applies if ``analyzer`` is not callable.	
	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	stop_words stop_words: {'english'}, list, default=None If 'english', a built-in stop word list for English is used. There are several known issues with 'english' and you should consider an alternative (see :ref:`stop_words`). If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if ``analyzer == 'word'``. If None, no stop words will be used. In this case, setting `max_df` to a higher value, such as in the range (0.7, 1.0), can automatically detect and filter stop words based on intra corpus document frequency of terms.	None
	token_pattern token_pattern: str or None, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp select tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the	'(?u)\\b\\w\\w+\\b'

	entire match, becomes the token. At most one capturing group is permitted.	
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different word n-grams or char n-grams to be extracted. All values of n such such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means only bigrams. Only applies if ``analyzer`` is not callable.	(1, ...)
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word n-gram or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``filename`` or ``file``, the data is first read from the file and then passed to the given callable analyzer.	'word'
	max_df max_df: float in range [0.0, 1.0] or int, default=1.0 When building the vocabulary ignore terms that have a docu-	1.0

	ment frequency strictly higher than the given threshold (corpus-specific stop words). If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	
	min_df min_df: float in range [0.0, 1.0] or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	max_features max_features: int, default=None If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used. This parameter is ignored if vocabulary is not None.	None
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents. Indices in the mapping should not be repeated and should not have any gap between 0 and the largest index.	None
	binary binary: bool, default=False	False

	If True, all non zero counts are set to 1. This is useful for discrete probabilistic models that model binary events rather than integer counts.	
	dtype dtype: dtype, default=np.int64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.int64'>

Name	Type	Value
fixed_vocabulary_ fixed_vocabulary_: bool True if a fixed vocabulary of term to indices mapping is provided by the user.	bool	False
vocabulary_ vocabulary_: dict A mapping of terms to feature indices.	dict	{'ab': 0, 'aback': 1, 'abaft': 2, 'abandon': 3, ...}

ab
aback
abaft
abandon
abandoned
abandoning
abandonment
abaout
abased
abasement
abashed
abashment
abate
abatement
abating
abbey
abbreviation
abbé
abdicated
abdication
abdications
abdomen

abdul
abernethy
aberrancy
aberrant
aberration
aberrations
abeyance
abhor
abhorred
abhorrence
abhorrent
abigail
abijah
abilities
ability
abject
abjure
ablaze
able
ably
abnormal
abnormalities
abnormality
abnormally
aboard
abode
abodes
abolished
abominable
abomination
abominations
aboriginal
abortion
abortions
abortive
abounds
about
above
abra
abreast
abroad
abrupt
abruptly

abruptness
absence
absences
absent
absolute
absolutely
absolved
absorb
absorbed
absorbing
absorbingly
absorption
abstain
abstemious
abstract
abstracted
abstraction
abstractions
abstractly
abstruse
abstruseness
absurd
absurdities
absurdity
absurdly
absurdness
absurdum
abundance
abundant
abundantly
abuse
abused
abuses
abutted
abutting

alpha	**alpha: float or array-like of shape (n_features,), default=1.0** **Additive (Laplace/Lidstone) smoothing parameter (set alpha=0 and force_alpha=True, for no smoothing).**	**1.0**

	force_alpha force_alpha: bool, default=True If False and alpha is less than 1e-10, it will set alpha to 1e-10. If True, alpha will remain unchanged. This may cause numerical errors if alpha is too close to 0. .. versionadded:: 1.2 .. versionchanged:: 1.4 The default value of `force_alpha` changed to `True`.	True
	fit_prior fit_prior: bool, default=True Whether to learn class prior probabilities or not. If false, a uniform prior will be used.	True
	class_prior class_prior: array-like of shape (n_classes,), default=None Prior probabilities of the classes. If specified, the priors are not adjusted according to the data.	None

Name	Type	Value
class_count_ class_count_: ndarray of shape (n_classes,) Number of samples encountered for each class during fitting. This value is weighted by the sample weight when provided.	ndarray[float64](3,)	[6320.,4508.,4835.]
class_log_prior_ class_log_prior_: ndarray of shape (n_classes,) Smoothed empirical log probability for each class.	ndarray[float64](3,)	[−0.91,-1.25,-1.18]
classes_ classes_: ndarray of shape (n_classes,) Class labels known to the classifier	ndarray[<U3](3,)	['EAP','HPL','MWS']
feature_count_ feature_count_: ndarray of shape (n_classes, n_features)	ndarray[float64](3, 22902)	[[1.,2.,0.,...,1.,0.,0.], [0.,0.,0.,...,0.,2.,1.], [0.,0.,1.,...,0.,0.,0.]]

Number of samples encountered for each (class, feature) during fitting. This value is weighted by the sample weight when provided.		
feature_log_prob_ feature_log_prob_: ndarray of shape (n_classes, n_features) Empirical log probability of features given a class, ``P(x_i|y)``.	ndarray[float64](3, 22902)	[[−11.38,-10.98,-12.08,...,-11.38,-12.08,-12.08], [−11.87,-11.87,-11.87,...,-11.87,-10.77,-11.18], [−11.92,-11.92,-11.23,...,-11.92,-11.92,-11.92]]
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	22902

On obtient une précision satisfaisante:

```
Précision: 0.8429519918283963
```

Les performances décomposées sont les suivantes:

```
              precision    recall  f1-score   support

         EAP       0.85      0.85      0.85      1580
         HPL       0.87      0.82      0.84      1127
         MWS       0.81      0.86      0.83      1209

    accuracy                           0.84      3916
   macro avg       0.84      0.84      0.84      3916
weighted avg       0.84      0.84      0.84      3916
```

Sans surprise, on obtient bien la prédiction de Mary Shelley:

```
np.str_('MWS')
```

Finalement, si on regarde les probabilités estimées (question 5), on se rend compte que la prédiction est très certaine:

	author	proba
0	EAP	0.001675
1	HPL	0.084844
2	MWS	0.913481

Comprendre la logique du classifieur naif de Bayes

Supposons que nous sommes dans un problème de classification avec des classes $(c_1,...,c_K) (ensemble noté $\mathcal{C}$). Nous plaçant dans le cadre de pensée du sac de mot, nous pouvons ne pas

nous préoccuper des positions des mots dans les documents, qui complexifieraient beaucoup l'écriture de nos équations.

L'équation Équation 2 peut être réécrite

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(w_1, ..., w_n|c)\mathbb{P}(c)$$

Dans le monde bayésien, on nomme $\mathbb{P}(w_1, ..., w_n|c)$ la vraisemblance (*likelihood*) et $\mathbb{P}(c)$ l'*a priori* (*prior*).

L'hypothèse Bayes naive permet de traiter un document comme une suite de tirages aléatoires dont les probabilités ne dépendent que de la catégorie. Dans ce cas, le tirage d'une phrase est une suite de tirages de mots et la probabilité composée est donc

$$\mathbb{P}(w_1, ..., w_n|c) = \prod_{i=1}^{n} \mathbb{P}(w_i|c)$$

Par exemple, en simplifiant en deux classes, si les probabilités sont celles du Table 1, la phrase *« afraid by Doctor Frankenstein »* aura un peu moins de 1% de chance (0.8%) d'être écrite si l'autrice est Mary Shelley mais sera encore moins vraisemblable chez Lovecraft (0.006%) car si *« afraid »* est très probable chez lui, Frankenstein est un événement rare qui rend peu vraisemblable cette composition de mots.

Table 1. – Exemple fictif de probabilités de tirage

Mot (w_i)	Probabilité chez Mary Shelley	Probabilité chez Lovecraft
Afraid	0.1	0.6
By	0.2	0.2
Doctor	0.2	0.05
Frankenstein	0.2	0.01

En composant ces différentes équations, on obtient

$$\hat{c} = \arg\max_{c \in \mathcal{C}} \mathbb{P}(c) \prod_{w \in \mathcal{W}} \mathbb{P}(w|c)$$

La contrepartie empirique de $\mathbb{P}(c)$ est assez évidente: la fréquence observée de chaque catégorie (les auteurs) dans notre corpus. Autrement dit,

$$\widehat{\mathbb{P}(c)} = \frac{n_c}{n_{doc}}$$

Quelle est la contrepartie empirique de $\mathbb{P}(w_i|c)$? C'est la fréquence d'apparition du mot en question chez l'auteur. Pour le calculer, il suffit de compter le nombre de fois qu'il apparaît chez l'auteur et de diviser par le nombre de mots de l'auteur.

4 Le modèle Word2Vec, une représentation plus synthétique

4.1 Vers une représentation plus synthétique du langage

La représentation vectorielle issue de l'approche *bag of words* n'est pas très synthétique ni stable et surtout est assez frustre.

Si on a un petit corpus, on va avoir des problèmes à extrapoler puisque de nouveaux textes ont toutes les chances d'apporter de nouveaux mots, qui sont de nouvelles dimensions de *features* qui n'étaient pas présentes dans le corpus d'entraînement, ce qui conceptuellement est un problème puisque les algorithmes de *machine learning* n'ont pas vocation à prédire sur des caractéristiques sur lesquelles ils n'ont pas été entraîné[1].

A l'inverse, plus on a de texte dans un corpus, plus notre représentation vectorielle sera importante. Par exemple, si votre sac de mot a vu tout le vocabulaire français, soit 60 000 mots selon l'Académie Française (les estimations étant de 200 000 pour la langue anglaise), cela fait des vecteurs de taille conséquente. Cependant, la diversité des textes est, en pratique, bien moindre: l'usage courant du Français nécessite plutôt autour de 3000 mots et la plupart des textes, notamment s'ils sont courts, n'utilisent pas un vocabulaire si complet. Ceci implique donc des vecteurs très peu denses, avec beaucoup de 0.

La vectorisation selon cette approche est donc peu efficace; le signal est peu compressé. Des représentations denses, c'est-à-dire de dimension plus faible mais portant toutes une information, semblent plus adéquate pour pouvoir généraliser notre modélisation du langage. L'algorithme qui a rendu célèbre cette approche est le modèle `Word2Vec`, en quelques sortes le premier ancêtre commun des LLM modernes. La représentation vectorielle de `Word2Vec` est assez synthétique: la dimension de ces *embeddings* est entre 100 et 300.

4.2 Des relations sémantique entre les termes

Cette représentation dense va représenter une solution à une limite de l'approche *bag of words* que nous avons évoquée à de multiples reprises. Chacune de ces dimensions va représenter un facteur latent, c'est à dire une variable inobservée, de la même manière que les composantes principales produites par une ACP. Ces dimensions latentes peuvent être interprétées comme des dimensions « fondamentales » du langage

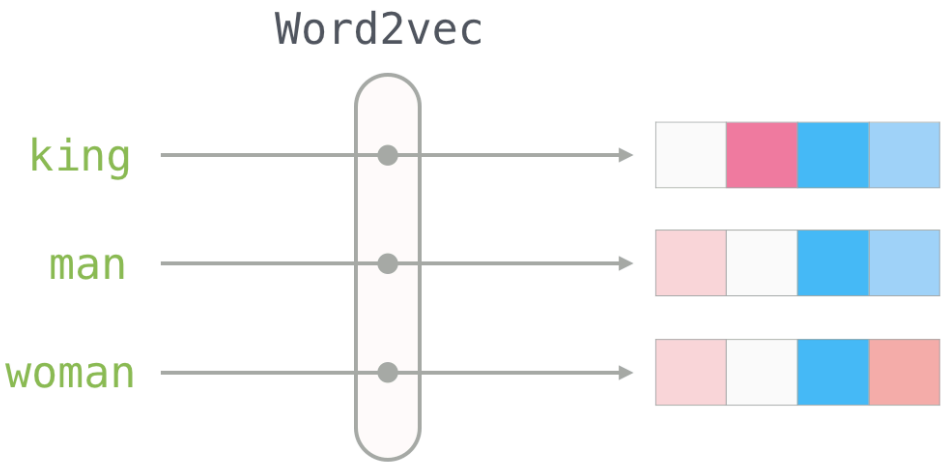


Figure 1. – Illustration du principe de la représentation de Word2Vec (source: Jay Alammar)

Par exemple, un humain sait qu'un document contenant le mot *« Roi »* et un autre document contenant le mot *« Reine »* ont beaucoup de chance d'aborder des sujets semblables. Un modèle `Word2Vec` bien entraîné va capter qu'il existe un facteur latent de type *« royauté »* et la similarité entre les vecteurs associés aux deux mots sera forte.

[1]Cette remarque peut apparaître étonnante alors que les IA génératives occupent une place importante dans nos usages. Néanmoins, il faut garder à l'esprit que certes vous posez de nouvelles questions à des IA mais vous les posez dans des termes qu'elles connaissent: du langage naturel dans une langue présente dans leur corpus d'entraînement, des images numériques qui sont donc interprétables par une machine, etc. Autrement dit, votre *prompt* n'est pas, en soi, inconnu pour l'IA, elle peut l'interpréter même si son contenu est nouveau et original.

La magie va même plus loin : le modèle captera aussi qu'il existe un facteur latent de type *« genre »*, et va permettre de construire un espace sémantique dans lequel les relations arithmétiques entre vecteurs ont du sens. Par exemple,

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

ou, pour reprendre, l'exemple issu du papier originel `Word2Vec` [1],

$$\text{Paris} - \text{France} + \text{Italy} \approx \text{Rome}$$

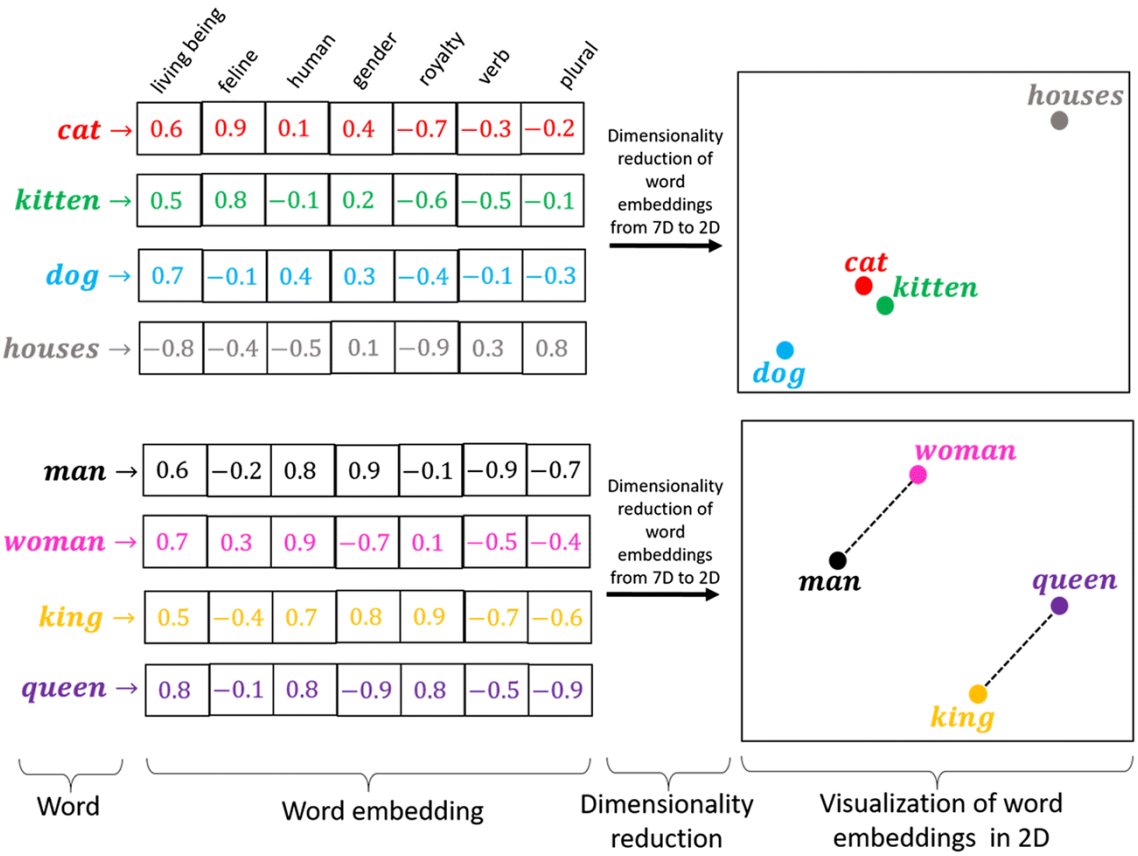


Figure 2. – Illustration du plongement lexical. Source : Post de blog Word Embedding : Basics

Un autre « miracle » de cette approche est qu'on obtient une forme de transfert entre les langues. Les relations sémantiques pouvant être similaires entre les langues, pour de nombreux mots usuels, on peut voir translater certaines langues les unes avec les autres si elles ont un socle commun (par exemple les langues occidentales). Ce concept est le point de départ des traducteurs automatiques et des IA multilingues

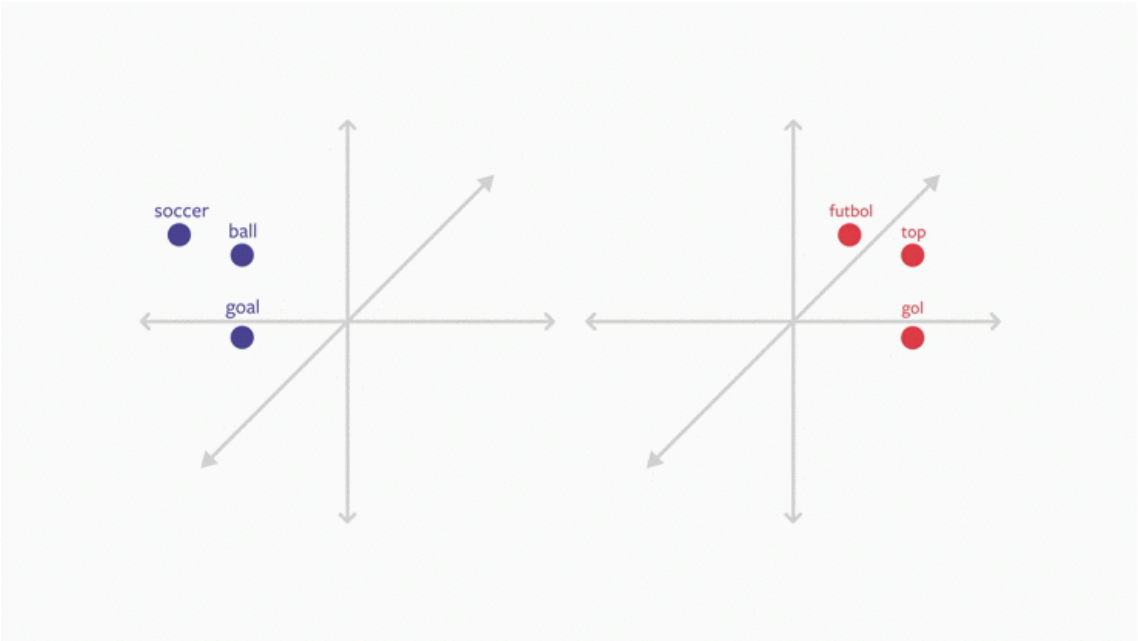


Figure 3. – Exemple de translation entre deux représentations vectorielles. Source: Meta

4.3 Comment ces modèles sont-ils entraînés ?

Ces modèles sont entraînés à partir d'une tâche de prédiction résolue par un réseau de neurones simple, généralement avec une approche par renforcement.

L'idée fondamentale est que la signification d'un mot se comprend en regardant les mots qui apparaissent fréquemment dans son voisinage. Pour un mot donné, on va donc essayer de prédire les mots qui apparaissent dans une fenêtre autour du mot cible.

En répétant cette tâche de nombreuses fois et sur un corpus suffisamment varié, on obtient finalement des *embeddings* pour chaque mot du vocabulaire, qui présentent les propriétés discutées précédemment. L'ensemble des articles `Wikipedia` est un des corpus de prédilection des personnes ayant construit des plongements lexicaux. Il comporte en effet des phrases complètes, contrairement à des informations issues de commentaires de réseaux sociaux, et propose des rapprochements intéressants entre des personnes, des lieux, etc.

Le contexte d'un mot est défini par une fenêtre de taille fixe autour de ce mot. La taille de la fenêtre est un paramètre de la construction de l'*embedding*. Le corpus fournit un grand ensemble d'exemples mots-contexte, qui peuvent servir à entraîner un réseau de neurones.

Plus précisément, il existe deux approches, dont nous ne développerons pas les détails :

- *Continuous bag of words* (CBOW), où le modèle est entraîné à prédire un mot à partir de son contexte ;
- *Skip-gram*, où le modèle tente de prédire le contexte à partir d'un seul mot.

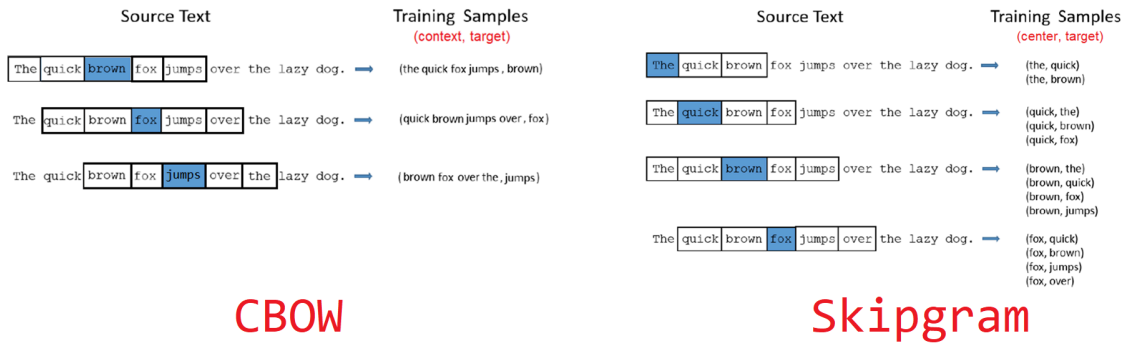


Figure 4. – Illustration de la différence entre les approches CBOW et Skip-gram

4.4 Modèles liés

Plusieurs modèles ont une filiation directe avec le modèle `Word2Vec` quoiqu'ils s'en distinguent par la nature de l'architecture utilisée.

C'est le cas, par exemple, du modèle modèle `GloVe`, développé en 2014 à Stanford, qui ne repose pas sur des réseaux de neurones mais sur la construction d'une grande matrice de co-occurrences de mots. Pour chaque mot, il s'agit de calculer les fréquences d'apparition des autres mots dans une fenêtre de taille fixe autour de lui. La matrice de co-occurrences obtenue est ensuite factorisée par une décomposition en valeurs singulières.

Le modèle `FastText`, développé en 2016 par une équipe de `Facebook`, fonctionne de façon similaire à `Word2Vec` mais se distingue particulièrement sur deux points :

- En plus des mots eux-mêmes, le modèle apprend des représentations pour les n-grams de caractères (sous-séquences de caractères de taille n, par exemple « *tar* », « *art* » et « *rte* » sont les trigrammes du mot « *tarte* »), ce qui le rend notamment robuste aux variations d'orthographe ;
- Le modèle a été optimisé pour que son entraînement soit particulièrement rapide.

Le modèle `FastText` est particulièrement performant pour les problématiques de classification automatique. L'Insee l'utilise par exemple pour plusieurs modèles de classification de libellés textuels dans des nomenclatures.

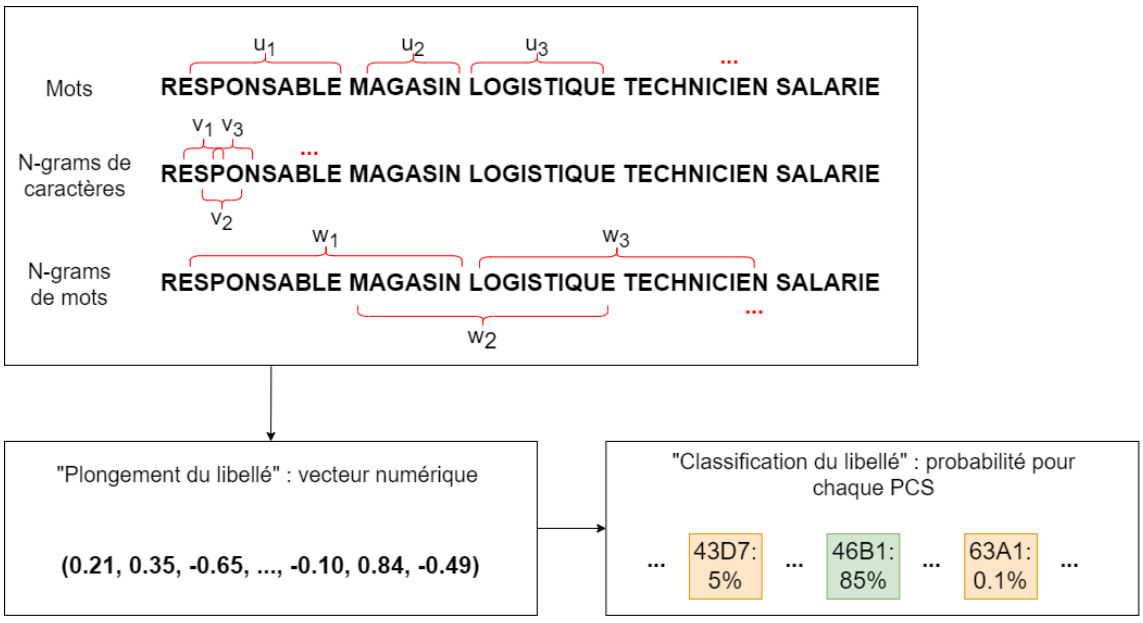


Figure 5. – Illustration du modèle fastText

Voici un exemple sur un projet de classification automatisée des professions dans la typologie des nomenclatures d'activités (les PCS) à partir d'un modèle entraîné par la librairie `Fasttext` :

```
Erreur lors de l'appel API : 503 Server Error: Service Unavailable for url: https://codification-ape-test.lab.sspcloud.fr/predict?nb_echos_max=3&prob_min=0&text_feature=data%20scientist
```

Ces modèles sont héritiers de `Word2Vec` dans le sens où ils reprennent une représentation vectorielle dense de faible dimension de documents textuels. `Word2Vec` reste un modèle héritier de la logique sac de mot. La représentation d'une phrase ou d'un document est une forme de moyenne des représentations des mots qui les composent.

Depuis 2013, plusieurs révolutions ont amené à enrichir les modèles de langage pour aller au-delà d'une représentation par mot de ceux-ci. Des architectures beaucoup plus complexes pour représenter non seulement les mots sous forme d'*embeddings* mais aussi les phrases et les documents sont aujourd'hui à l'oeuvre et peuvent être reliées à la révolution des architectures *transformers*.

5 Les *transformers*: une représentation plus riche du langage

Si le modèle `Word2Vec` est entraîné de manière contextuelle, sa vocation est de donner une représentation vectorielle d'un mot de manière absolue, indépendamment du contexte. Par exemple, le terme *« banc »* aura exactement la même représentation vectorielle qu'il se trouve dans la phrase *« Elle court vers le banc de sable »* ou « Il t'attend sur un banc au parc »_. C'est une limite majeure de ce type d'approche et on se doute bien de l'importance du contexte pour l'interprétation du langage.

L'objectif des architectures *transformers* est de permettre des représentations vectorielles contextuelles. Autrement dit, un mot aura plusieurs représentations vectorielles, selon son contexte d'occurrence. Ces modèles s'appuient sur le mécanisme d'attention [2]. Avant cette approche, lorsqu'un modèle apprenait à vectoriser un texte et qu'il arrivait au énième mot, la seule mémoire qu'il gardait était celle du mot précédent. Par récurrence, cela signifiait qu'il gardait une mémoire des mots précédents mais celle-ci tendait à se dissiper. Par conséquent, pour un mot arrivant loin dans la phrase, il était probable que le contexte de début de phrase était oublié. Autrement dit, dans la phrase *« à la plage, il allait explorer le banc »*, il était fort probable qu'arrivé au mot *« banc »*, le modèle ait oublié le début de phrase qui avait pourtant de l'importance pour l'interprétation.

L'objectif du mécanisme d'attention est de créer une mémoire interne au modèle permettant, pour tout mot d'un texte, de pouvoir garder trace des autres mots. Bien-sûr tous ne sont pas pertinents pour interpréter le texte mais cela évite d'oublier ceux qui sont importants. L'innovation principale des dernières années en NLP a été de parvenir à créer des mécanismes d'attention à grande échelle sans pour autant rendre intractables les modèles. Les fenêtres de contexte des modèles les plus performants deviennent immenses. Par exemple le modèle Llama 3.1 (rendu public par Meta en Juillet 2024) propose une fenêtre de contexte de 128 000 *tokens*, soit environ 96 000 mots, l'équivalent du *Hobbit* de Tolkien. Autrement dit, pour déduire la subtilité du sens d'un mot, ce modèle peut parcourir un contexte aussi long qu'un roman d'environ 300 pages.

Les deux modèles qui ont marqué leur époque dans le domaine sont les modèles `BERT` développé en 2018 par *Google* (qui était déjà à l'origine de `Word2Vec`) et la première version du bien-connu `GPT` d'`OpenAI`, qui, en 2017, était le premier modèle préentraîné basé sur l'architecture *transformer*. Ces deux familles de *transformer* diffèrent dans la manière dont ils intègrent le contexte pour faire une prédiction. `GPT` est un modèle autorégressif, donc ne considère que les *tokens* avant celui dont on désire faire une prédiction. `BERT` utilise les *tokens* à gauche et à droite pour inférer le contexte. Ces

deux grands modèles de langage entraînés sont entraînés par auto-renforcement, principalement sur des tâches de prédiction du prochain *token* [3]. Depuis le succès de `ChatGPT`, les nouveaux modèles GPT (à partir de la version 3) ne sont plus *open source*. Pour les utiliser, il faut donc passer par les API d'OpenAI. Il existe néanmoins de nombreuses alternatives dont les poids sont ouverts, à défaut d'être *open source*[2], qui permettent d'utiliser ces LLM par le biais de `Python`, par le biais, notamment, de la librairie `transformers` développée par *Hugging Face*.

Quand on travaille avec des corpus de taille restreinte, c'est généralement une mauvaise idée d'entraîner son propre modèle *from scratch*. Heureusement, des modèles pré-entraînés sur de très gros corpus sont disponibles. Ils permettent de réaliser du *transfer learning*, c'est-à-dire de bénéficier de la performance d'un modèle qui a été entraîné sur une autre tâche ou bien sur un autre corpus.

Exercice 3

1. Refaire un train/test split avec 500 lignes aléatoires
2. Importer le modèle `all-MiniLM-L6-v2` avec le package `sentence transformers`. Encoder `X_train` et `X_test`
3. Faire une classification avec une méthode simple, par exemple des SVC, s'appuyant sur les *embeddings* produits à la question précédente. Comme le jeu d'entraînement est réduit, vous pouvez faire de la validation croisée.
4. Comprendre pourquoi les performances sont détériorées par rapport au classifieur naif de Bayes.

Réponse à la question 1:

```
random_rows = spooky_df.sample(500)
y = random_rows["author"]
X = random_rows['text']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Réponse à la question 2:

```
from sentence_transformers import SentenceTransformer
from sklearn.svm import LinearSVC

model = SentenceTransformer(
    "all-MiniLM-L6-v2", model_kwargs={"torch_dtype": "float16"}
)

X_train_vectors = model.encode(X_train.values)
X_test_vectors = model.encode(X_test.values)
```

Réponse à la question 3:

```
from sklearn.model_selection import cross_val_score

clf = LinearSVC(max_iter=10000, C=0.1, dual="auto")
```

[2]Certaines organisations, comme Meta pour Llama, mettent à disposition les poids après entraînement de leur modèle sur la plateforme *Hugging Face*, permettant une réutilisation de ces modèles si la licence le permet. Néanmoins, il ne s'agit pas pour autant de modèles *open source* puisque le code utilisé pour entraîner les modèles et constituer les corpus d'apprentissage, issus de collectes massives de données par *webscraping*, et les éventuelles annotations supplémentaires pour en faire des versions spécialisées, ne sont pas partagés.

```
scores = cross_val_score(
    clf, X_train_vectors, y_train,
    cv=4, scoring='f1_micro', n_jobs=4
)

print(f"CV scores {scores}")
print(f"Mean F1 {np.mean(scores)}")
```

Mais pourquoi, avec une méthode très compliquée, ne parvenons-nous pas à battre une méthode toute simple ?

On peut avancer plusieurs raisons :

- le `TF-IDF` est un modèle simple, mais toujours très performant (on parle de *« tough-to-beat baseline »*).
- la classification d'auteurs est une tâche très particulière et très ardue, qui ne fait pas justice aux *embeddings*. Comme on l'a dit précédemment, ces derniers se révèlent particulièrement pertinents lorsqu'il est question de similarité sémantique entre des textes (*clustering*, etc.).

Dans le cas de notre tâche de classification, il est probable que certains mots (noms de personnage, noms de lieux) soient suffisants pour classifier de manière pertinente, ce que ne permettent pas de capter les *embeddings* qui accordent à tous les mots la même importance.

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
```

```
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie

[1] T. Mikolov, « Efficient estimation of word representations in vector space », *arXiv preprint arXiv:1301.3781*, vol. 3781, 2013.

[2] A. Vaswani, « Attention is all you need », *Advances in Neural Information Processing Systems*, 2017.

[3] « The Hugging Face Course, 2022 ». 2022.